STRINGS II
ISTA 130

TRAVERSING BY INDEX VS. BY ELEMENT

```
# traverse by index
for i in range(len(word)):
    print(word[i])

# traverse by element
for ch in word:
    print(ch)
```

- These loops do exactly the same thing – `word[i] == ch`
- Putting the elements of a sequence, in this case a string, directly into a loop control variable is a convenience Python offers.

TRAVERSING BY INDEX VS. BY ELEMENT

- You can always traverse by index.
- But sometimes traversing by element won't work:

```
# traverse by index
limit = min(len(word1), len(word2))
for i in range(limit):
    print(word1[i], '-', word2[i])
```

```
# traverse by element
for ch in word1:
    # ??????
```

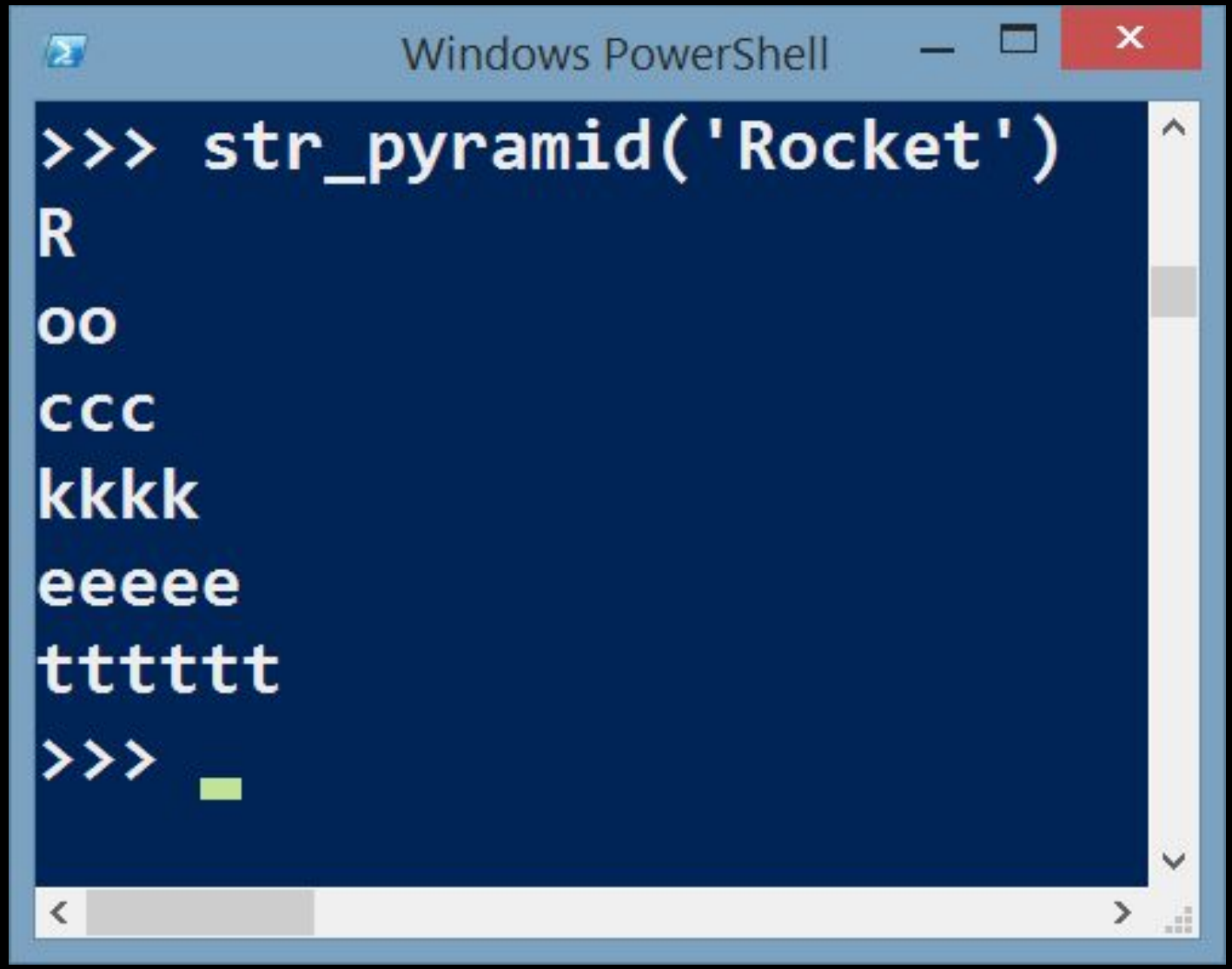
Traverse By Index vs. By Element

- If there is a call to the `range` function, you are traversing by index and your loop control variable is an integer. The loop control variable is used to access elements of the sequence by indexing, e.g. `string[i]`. You can also do other things with the index in the loop. ***You can always traverse by index.***
- Traversing by element stores the elements themselves in the loop control variable. You have no access to their indices. ***You CANNOT always traverse by element because sometimes you need the index inside the loop.*** It is a Python convenience that is not always appropriate.

String `s`: Traverse by Index vs. Traverse by Element

Traverse by Index	Traverse by Element
`for i in range(len(s)):` `  # do something with s[i]`	`for ch in s:` `  # do something with ch`
Loop control variable `i` is assigned `0, 1, 2, …, len(s) - 1`	Loop control variable `ch` is assigned `s[0], s[1], …, s[len(s) - 1]`
Loop control variable `i` type is `int`	Loop control variable `ch` type is `str`
Can always traverse by index	Won't work if you need to use the index in your loop (because you don't have it – you only have the characters)
Who indexes the string: you do	Who indexes the string: Python does
Call to the `range` function	No call to the `range` function

Can We Traverse By Element?



`str` is a ***Sequence*** Type

- Each letter is an ***element*** of the sequence.
- Each element has a ***position*** in the string.
- We number the positions in a string using integers starting at `0`. These numbers are called ***indices***. Each position has its own ***index***.
- When ***traversing by index***, the indices (integers) go into the loop control variable.
- When ***traversing by element***, the characters themselves go into the loop control variable.

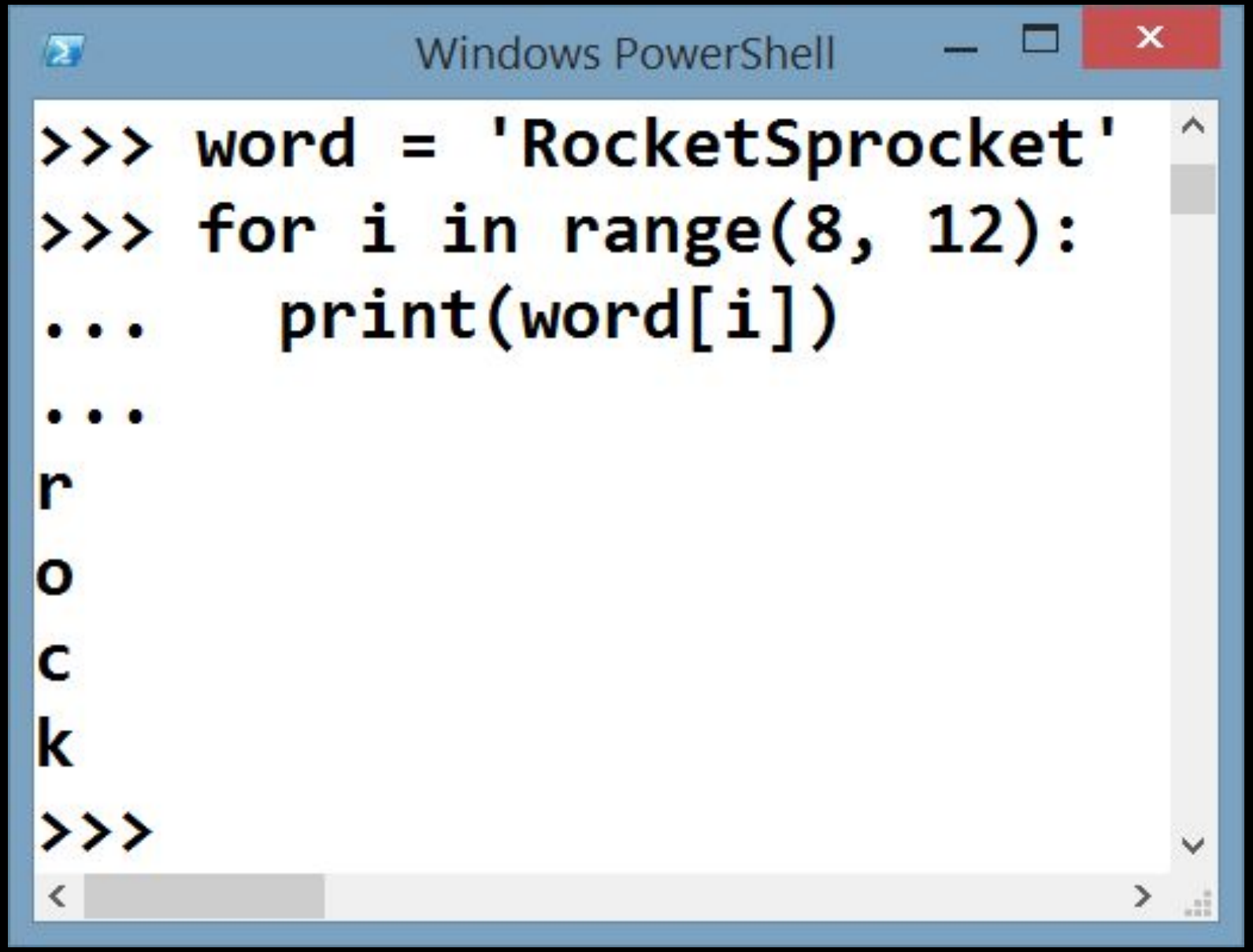
`for` vs. `while`

- Any `for` loop can be converted to a `while` loop.
- Any `while` loop that traverses a sequence must traverse by indexing – the user must initialize the index variable, use it in a condition to control the loop, and increment it (or decrement) it appropriately so that the loop ends at some point.
- Many `while` loops cannot be converted to `for`.

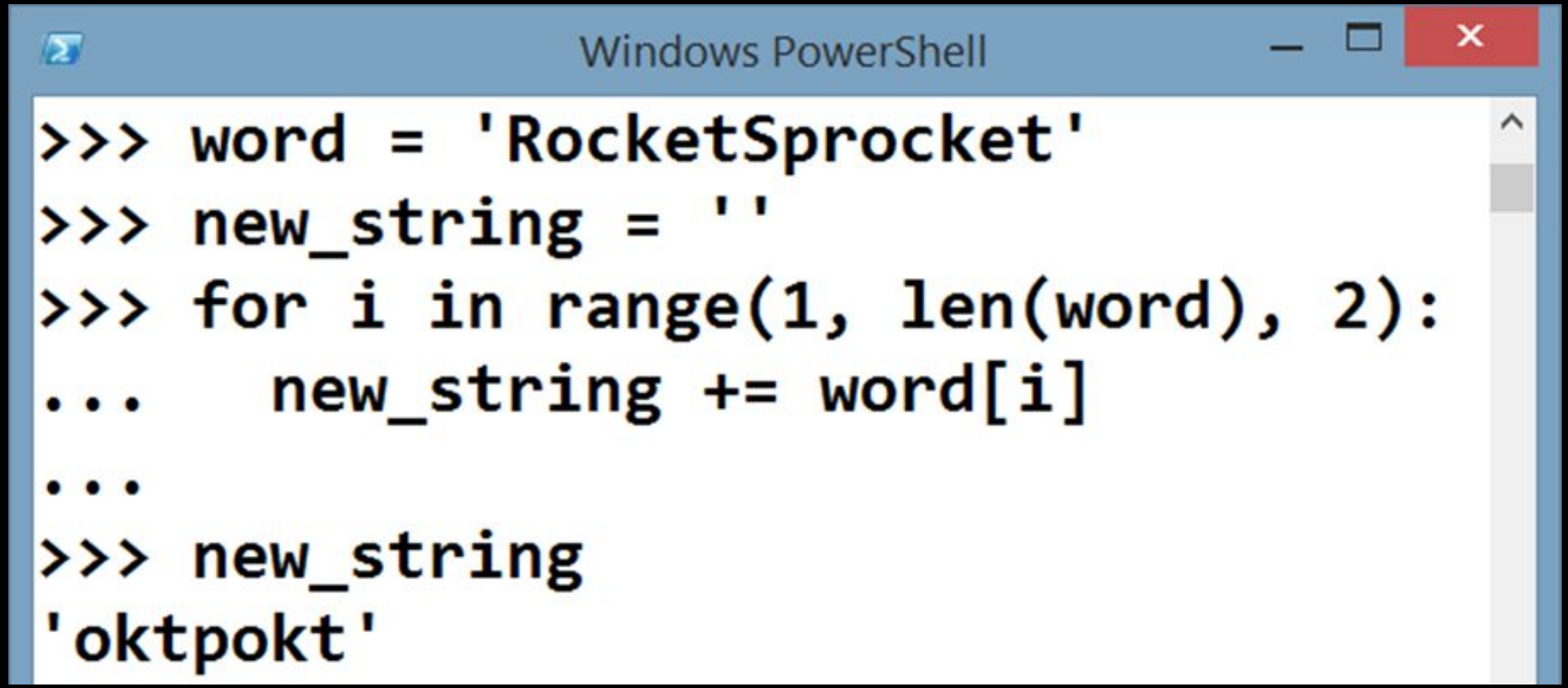
Sequence Looping In Python

- When you can:
  - Create a `for` loop that traverses by index
  - Create a `for` loop that traverses by element, if appropriate
  - Create a `while` loop that traverses by index
  - Convert any `for` loop to `while`
  - Convert any `while` to `for`, if possible
- You have pretty much mastered looping in Python.

`range` With Two Arguments



`range` With Three Arguments



+ index	0	1	2	3	4	5	6
character	b	o	l	o	g	n	a
- index	-7	-6	-5	-4	-3	-2	-1

Regular ASCII Chart (character codes 0 - 127)

000		(nul)	016	►	(dle)	032	sp	048	0	064	@	080	P	096	`	112	p
001	☺	(soh)	017	◄	(dc1)	033	!	049	1	065	A	081	Q	097	a	113	q
002	☻	(stx)	018	↕	(dc2)	034	"	050	2	066	B	082	R	098	b	114	r
003	♥	(etx)	019	‼	(dc3)	035	#	051	3	067	C	083	S	099	c	115	s
004	♦	(eot)	020	¶	(dc4)	036	$	052	4	068	D	084	T	100	d	116	t
005	♣	(enq)	021	§	(nak)	037	%	053	5	069	E	085	U	101	e	117	u
006	♠	(ack)	022	▬	(syn)	038	&	054	6	070	F	086	V	102	f	118	v
007	•	(bel)	023	↨	(etb)	039	'	055	7	071	G	087	W	103	g	119	w
008	◘	(bs)	024	↑	(can)	040	(	056	8	072	H	088	X	104	h	120	x
009		(tab)	025	↓	(em)	041	)	057	9	073	I	089	Y	105	i	121	y
010		(lf)	026		(eof)	042	*	058	:	074	J	090	Z	106	j	122	z
011	♂	(vt)	027	←	(esc)	043	+	059	;	075	K	091	[	107	k	123	{
012	♀	(np)	028	∟	(fs)	044	,	060	<	076	L	092	\	108	l	124	|
013		(cr)	029	↔	(gs)	045	-	061	=	077	M	093	]	109	m	125	}
014	♫	(so)	030	▲	(rs)	046	.	062	>	078	N	094	^	110	n	126	~
015	☼	(si)	031	▼	(us)	047	/	063	?	079	O	095	_	111	o	127	⌂

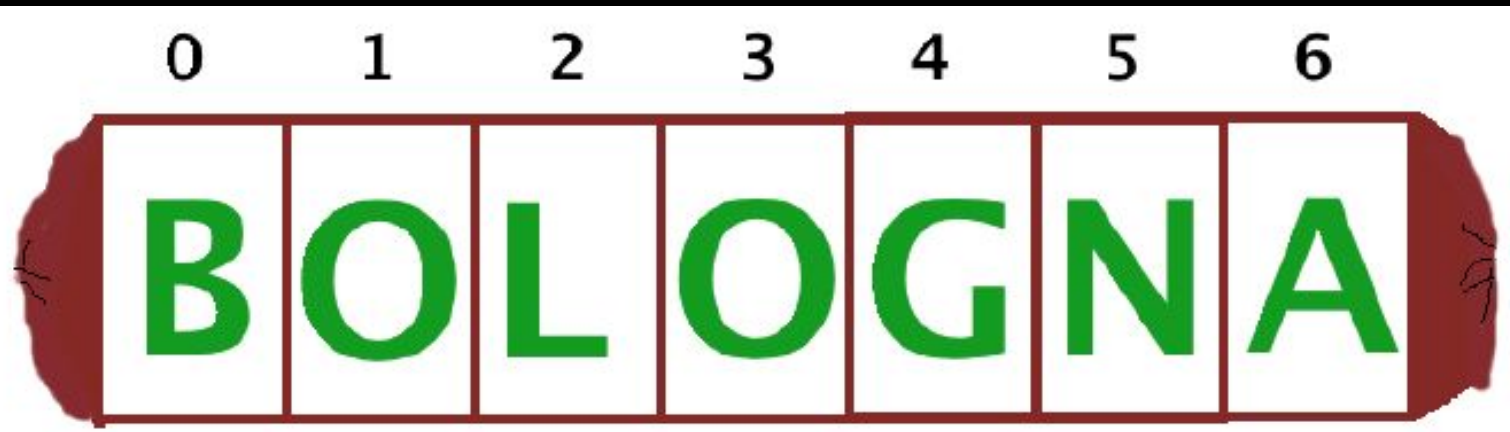
http://vcampus.co/blogs/5190/ascii-table-with-html-entity

Slicing `'bologna'`

- Sometimes we're interested in more than one character
- We'd like a thicker slice of "bologna"
- A *substring* is a part of a string
  - we need to tell `Python` where to start and where to end
  - `Python` uses *slice* notation to get a substring
- So far we've used the index operator like this: `word[n]`
- To get a slice we add a second integer: `word[n:m]`
  - the `n` is the start of the slice and the `m` is the end
  - the character at `m` is *not* included in the slice
- Try these:

```
word = 'bologna'
print(word[0:0])
print(word[0:1])
print(word[0:2])
```

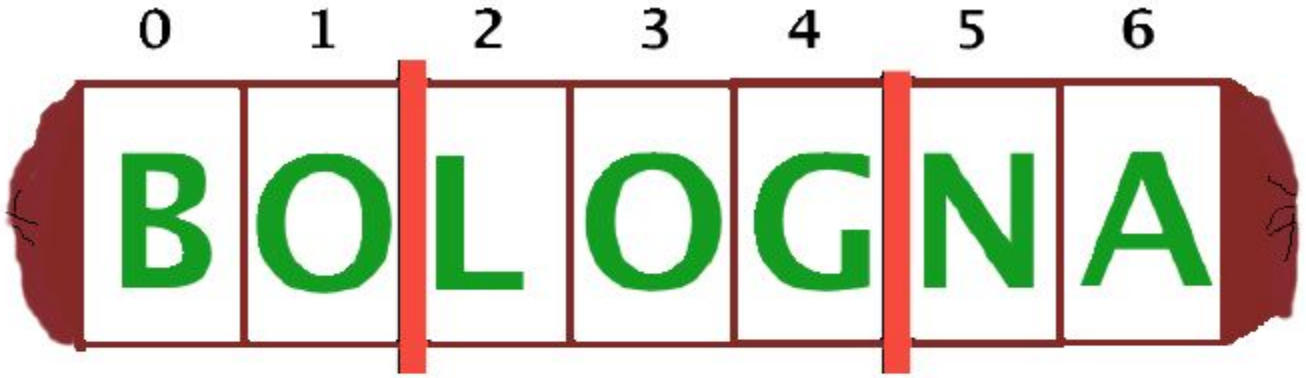
Slicing `'bologna'`



- Think of the indices as *offsets* from the beginning of the string (or the bologna).

```
word = 'bologna'
# The following line prints the character with 0
offset from the start
print(word[0])
# The following line prints the character offset 1
from the start
print(word[1])
```

Slicing `'bologna'`



- Now we take a slice of bologna:

```
word = 'bologna'
print(word[2:5])
```

- We're telling Python where to place the knife for each of the two cuts we make into the bologna.
- The character found at offset 5 is **NOT** part of our slice of bologna

Make the first cut *before* position 2 and the second *before* position 5.

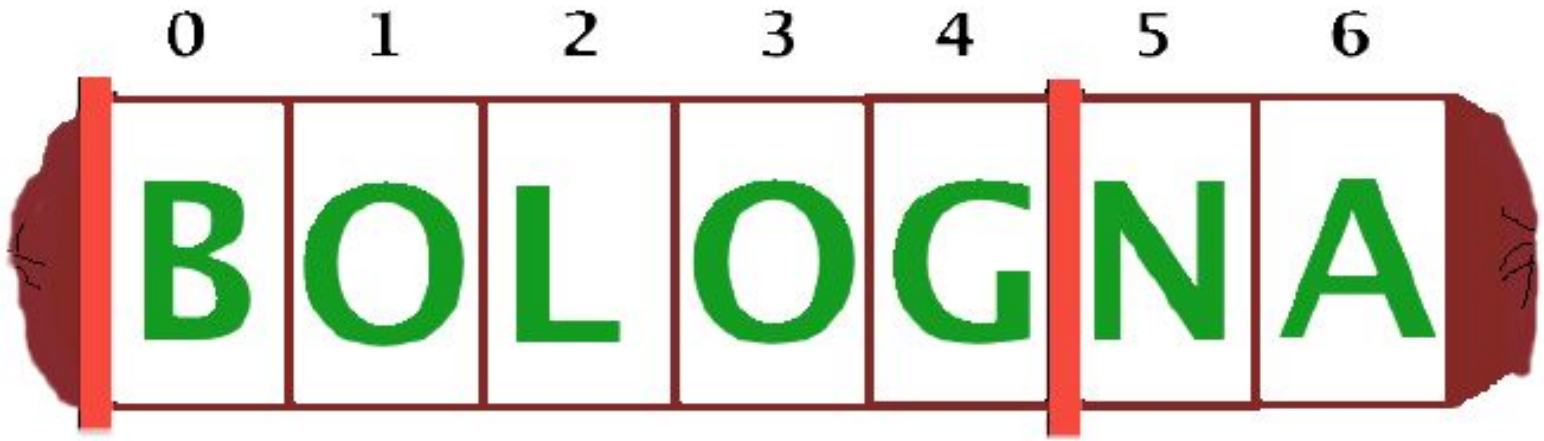
Slicing `'bologna'`

- We're telling Python where to place the knife for each of the two cuts we make into the bologna.
- In the slice `word[n:m]`, the character found at index `m` is **NOT** part the result.

```
word = 'bologna'
```

  - `print(word[0:3])` → `bol`
  - `print(word[1:3])` → `ol`
  - `print(word[2:3])` → `l`
  - `print(word[3:3])` →
  - `print(word[4:3])` →
    - Python will let you do things that don't make sense
  - `print(word[3:5])` → `og`

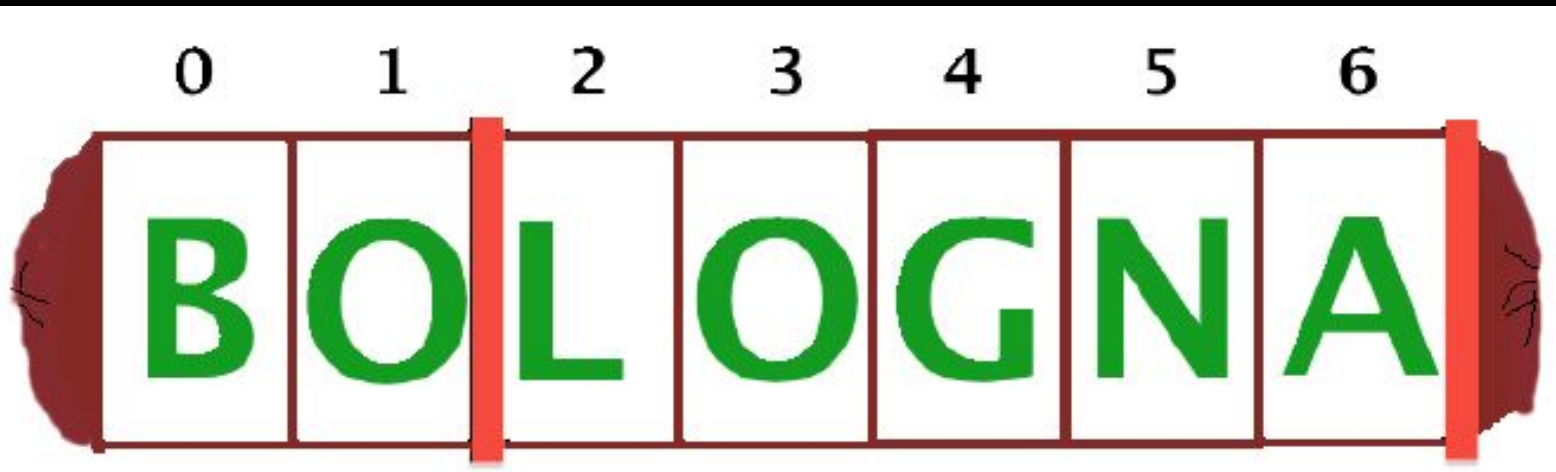
Slicing Shortcuts



- We can **omit the first index** in the slice and Python will assume we meant start at 0:
  - we could think of this as skipping the first cut into the bologna

```
word = 'bologna'
print(word[:5])
```

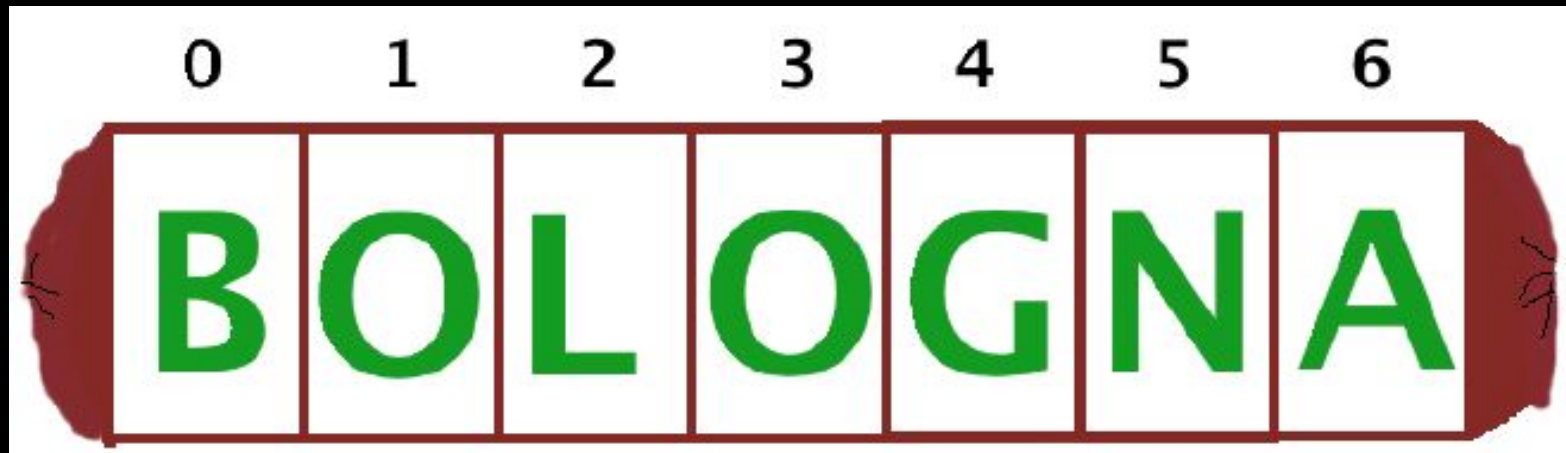
Slicing Shortcuts



- We can **omit the last index** in the slice and Python will assume we meant go to the end:
  - we could think of this as skipping the last cut into the bologna

```
word = 'bologna'
print(word[2:])
```

Slicing

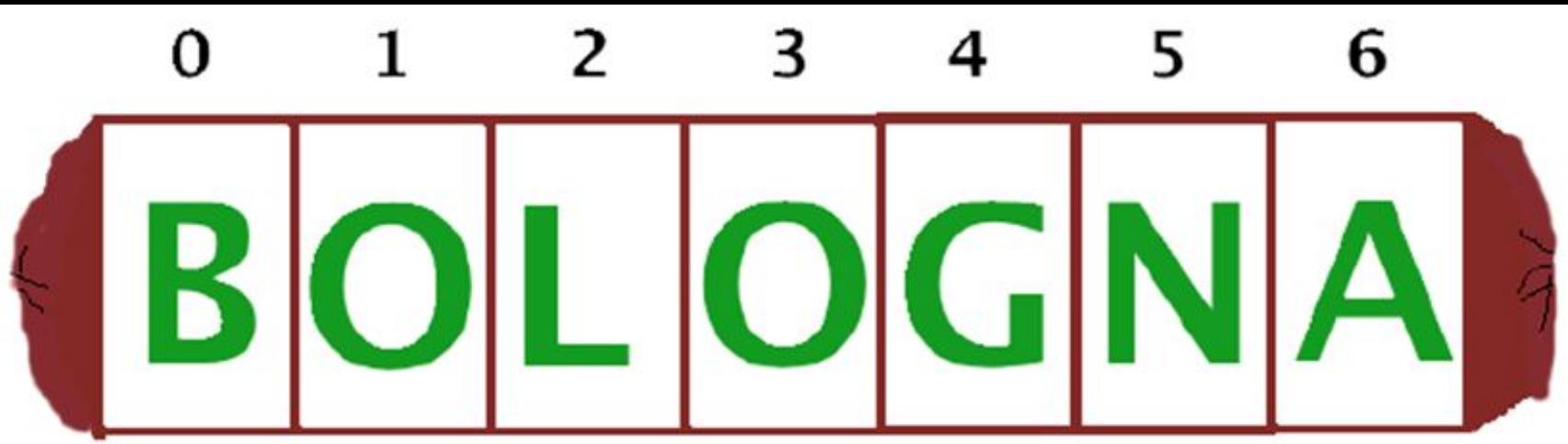


- What do you think this will do?

```
word = 'bologna'
print(word[:])
```

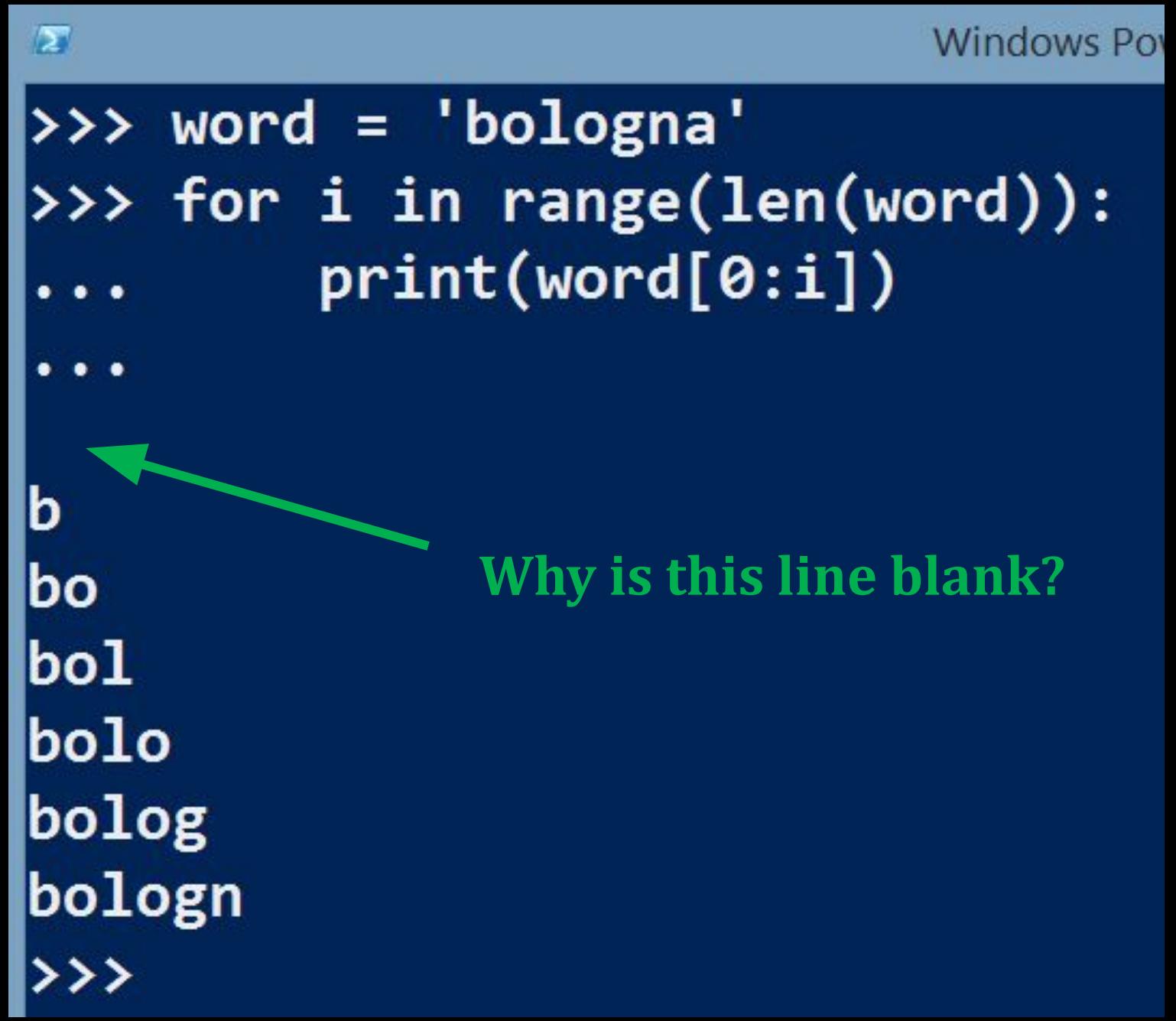
What about `print(word[1:-1])`?

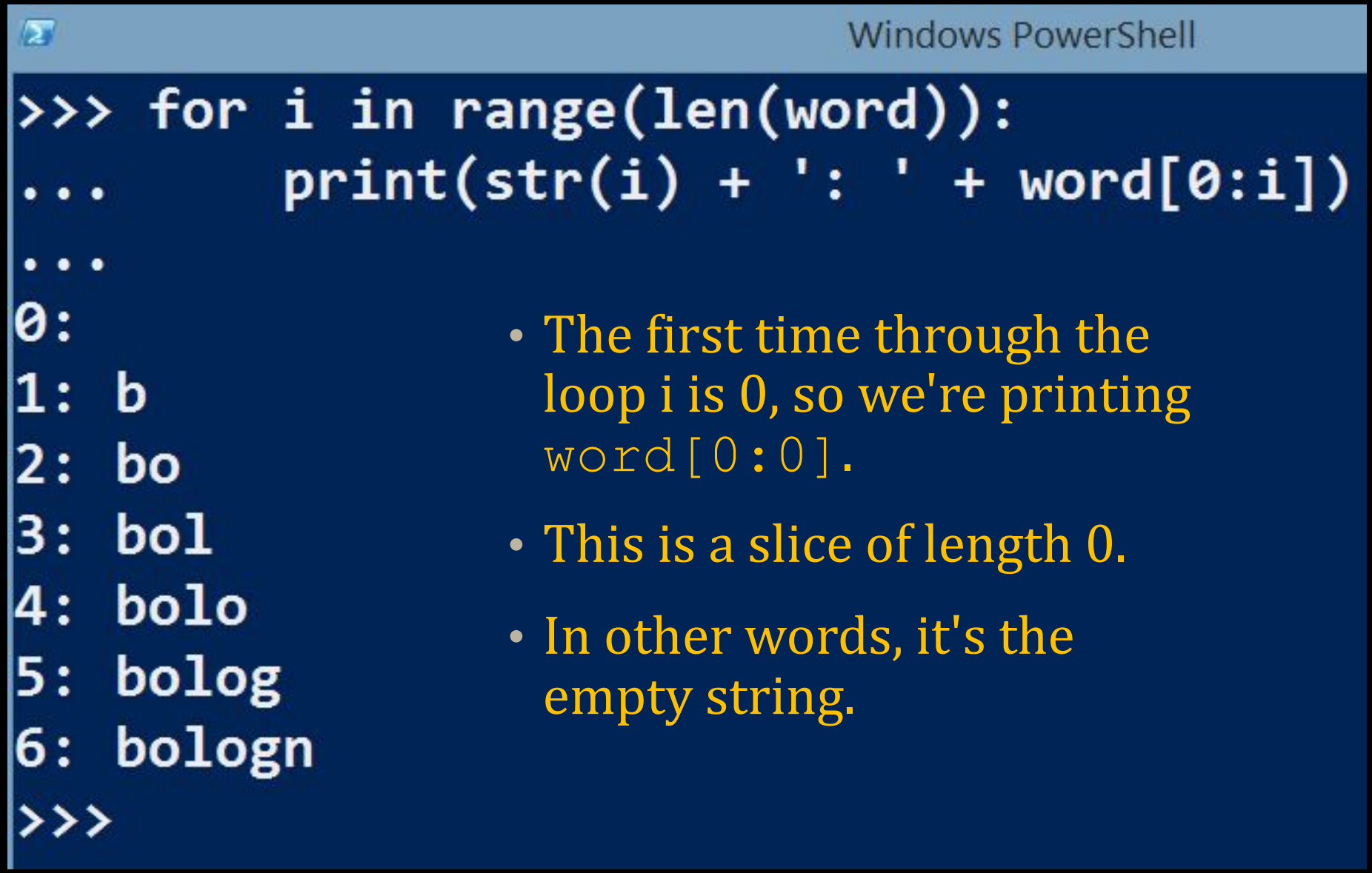
Slices In A Loop



- What will this do?

```
word = 'bologna'
for i in range(len(word)):
    print(word[0:i])  #
try changing to i + 1
```


Windows Po
>>> word = 'bologna'
>>> for i in range(len(word)):
...   print(word[0:i])
...
b
bo
bol
bolo
bolog
bologn
>>>
Why is this line blank?

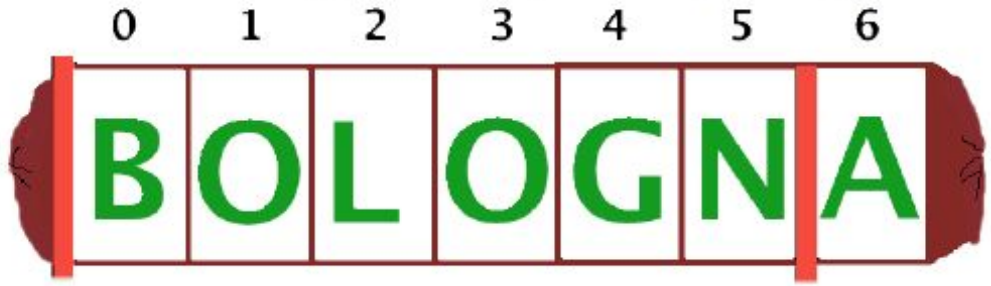

Windows PowerShell
>>> for i in range(len(word)):
... print(str(i) + ': ' + word[0:i])
...
0:
1: b
2: bo
3: bol
4: bolo
5: bolog
6: bologn
>>>
• The first time through the loop i is 0, so we're printing word[0:0].
• This is a slice of length 0.
• In other words, it's the empty string.

What About The Last Time Through?

Windows PowerShell

```
>>> for i in range(len(word)):
...     print(str(i) + ': ' + word[0:i])
...
0:
1: b
2: bo
3: bol
4: bolo
5: bolog
6: bologn
>>>
```

- The last time through the loop we're printing `word[0:6]`

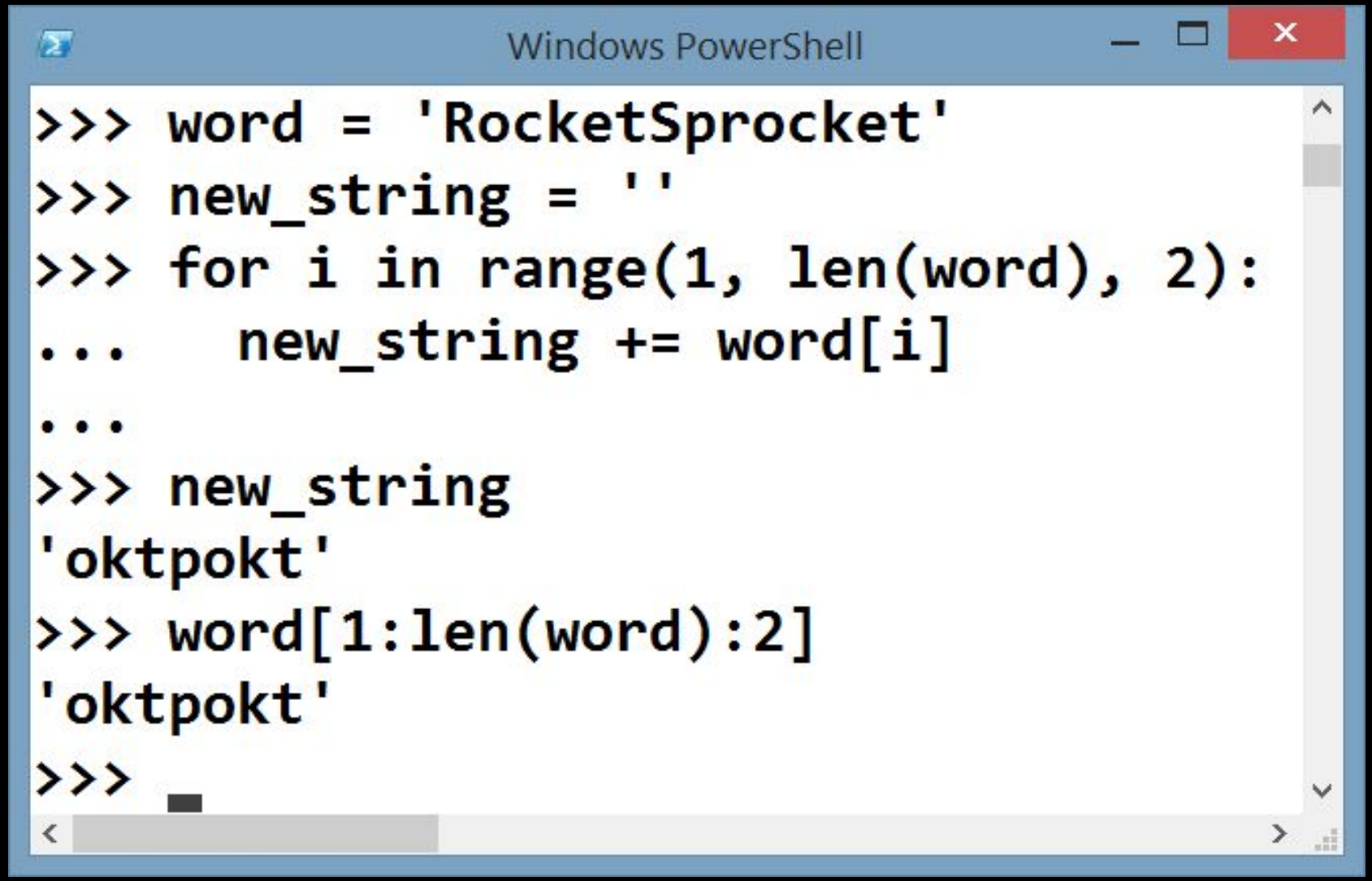


- To get the whole word we need
  `word[0:7]`
- If we want to see 'b' through 'bologna' what could we change?

Indexing vs. Slicing

- `word[len(word)]` is an error.
- `word[:len(word)]` is not an error.

`for` Loop vs. Slicing



Same as `word[1::2]`

String Methods

- To use a string *method* you use *dot notation*:

```
<the_string>.<the_method_name>
```

- For example, to tell the string "BOLOGNA" to call its `lower()` method do this:

```
'BOLOGNA'.lower()
```

- To tell the string "rabbits" to call its `upper()` method do this:

```
'rabbits'.upper()
```

- To have the string that the `name` variable currently refers to call its `upper()` method do this:

```
name = 'Ziggy'
name.upper()
```

String Methods

- Now try this:

```
name = 'Ziggy'
print(name.upper())
print(name)
```

- The `upper` and `lower` methods *do not* modify the original string, they return *new* strings

String Comparisons

- **Beware**: all uppercase characters come *before* all lowercase characters.[1]
- Try these in a shell:

```
"beer" == "Beer"
"beer" < "Beer"
"a" < "Z"
```

- What could you do to make these comparisons come out the way you expect them to?
- Use `str.lower()` (or `str.upper()`) on all strings before comparing them.

[1] Comparisons are based on character encoding scheme (e.g. ASCII). For more see `ord` and `chr` functions at: `http://docs.python.org/3.3/library/functions.html#chr`

String Methods

- Let's look at another method:

  `find`

- You can get help on string methods like this:

  ```
  help(str.find)
  ```

- The `find` method searches for a given substring and returns the index of the first occurrence of the substring
  - if the substring is not found it returns -1

- Try this:

  ```
  word = 'bologna'
  index_of_log = word.find('log')
  print(index_of_log)
  ```

- This tells us that we can find the string `'log'` beginning at index 2 of the string `word`

String Methods

- Try this:

```
word = 'bologna'
index_of_log = word.find('leg')
print(index_of_log)
```

- The string `'leg'` was *not* found in the string `word`

String Methods

- Try this:

```
word = 'BolognaBologna'
index_of_log = word.find('log', 3, 12)
print(index_of_log)
```

- When searching between 3 and 12, the string `'log'` is found at index 9 of the string `word`

String Methods

Replacing parts of a string:

- `replace(old, new , count )`

We aren't really replacing – the `replace` method returns a new string

String Methods

Making prettier output:

- `center(width)`
- `ljust(width)`
- `rjust(width)`

Times Table

String Methods

Here are some useful string methods:

- `str.capitalize()`
- `str.count(sub, start, end)`
- `str.find(sub, start, end)`
- `str.lower()`
- `str.upper()`
- `str.replace(old, new, count)`
- `str.strip()`
- `str.isalnum()`
- `str.isalpha()`
- `str.isdigit()`
- `str.isspace()`
- `str.center(width)`
- `str.ljust(width)`
- `str.rjust(width)`

  - You should look at `help` on each of these to see more details:
    - e.g. `help(str.count)`
    - Try experimenting with each method
  - See more string methods here:

    `http://docs.python.org/3.3/library/stdtypes.html#string-methods`

Worksheet – remember your string toolbox: concatenation, repetition, indexing, slicing, methods